Programming and Databases

Joern Ploennigs

Software Design

MIDJOURNEY: WATERFALL IN THE CHINESE MOUNTAIN RANGE

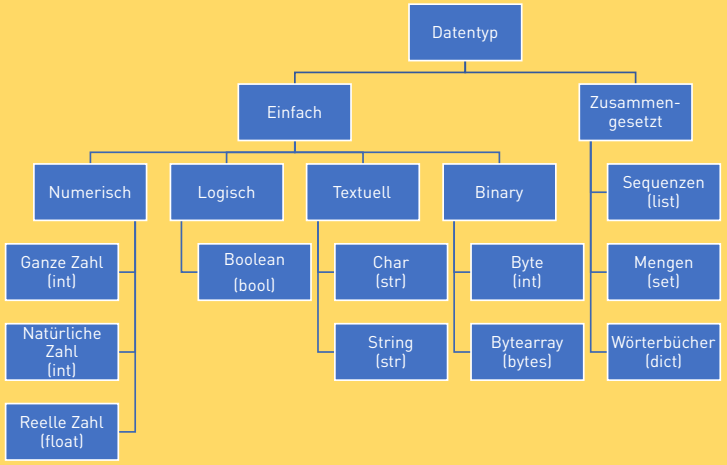
Recap: In-Class Question

What data types are there in Python?

Midjourney: A python programming a robot

Recap: Data Types - Python

Python simplifies some data types.



Recap: Lecture Hall Question

What are objects? Generalization, encapsulation, polymorphism?

Midjourney: Object-oriented man

Recap: Object-Oriented Programming

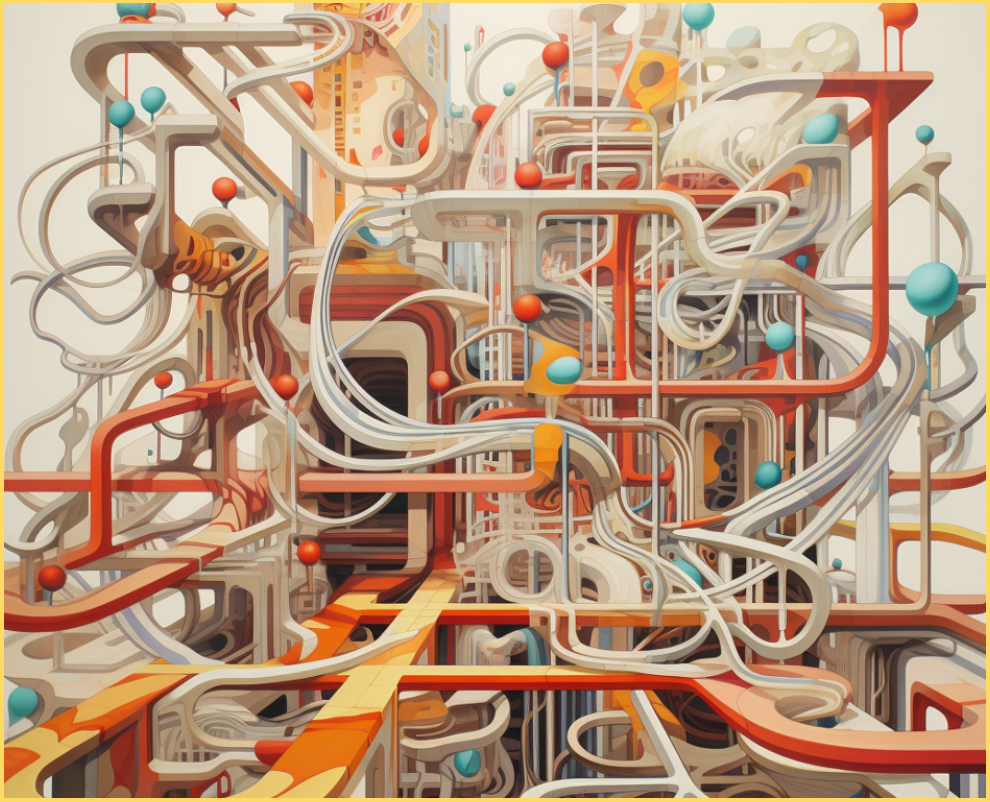
> 📘 **Definition: Object-Oriented Programming**
> Object-Oriented Programming (OOP) is a programming paradigm that assumes that a program consists exclusively of objects that interact cooperatively with one another.

Each object has:

- *Attributes* (Properties): Defines the value of the object's state.
- *Methods* define the possible state changes (actions) of an object.

Recap: In-Class Question

What are the four fundamental elements of a program?



Midjourney: Fundamentals of a Programming Language

Recap: Program Elements

- We all know the fundamental building blocks of a program:
  - Statements
  - Functions
  - Branching
  - Loops & Recursion
- We know how to design algorithms and combine these elements.

PROCEDURE

Modeling with Objects

- Our perception and our thinking rely on recognizing objects. This requires that we can separate an object from other objects (e.g., a chair and a table).
- Replicating this perception and this understanding of the world in a computer is called *Modeling*.
- Since our perception is based on objects, it is natural to use objects for modeling as well.
- The objects in the world interact; we must also model this behavior.

📘 **Definition: Model**

A model is an abstract, simplified representation of a system (usually from reality).

Lecture Hall Question

What do these elements have in common?

Lecture Hall Question

- These are all *geometric objects*
- They all consist of *points* that are connected by *lines*
- We can leverage this in modeling and define classes so that they exploit the *commonality*

Object-Oriented Software Design

Definition: Object-Oriented Software Design

In object-oriented software design, a program is designed to consist solely of *objects*.

- The software design is closely mirrored by the approach engineers use when designing plans.
- The usual modeling language is *UML diagrams*.
- One models in the design how:
  - objects in the form of *classes* are defined,
  - which *attributes and methods* they possess,
  - how these classes build upon one another (*inheritance*),
  - as well as how they stand in static relationships with each other (*references*)
  - and how they interact dynamically (*behavior*)

Referenzen

- In programs, objects are usually related to one another. For example, a line consists of two points.
- This relationship between two object classes is called a *reference*.
- References in Python are created as attributes whose type is that of the other object.

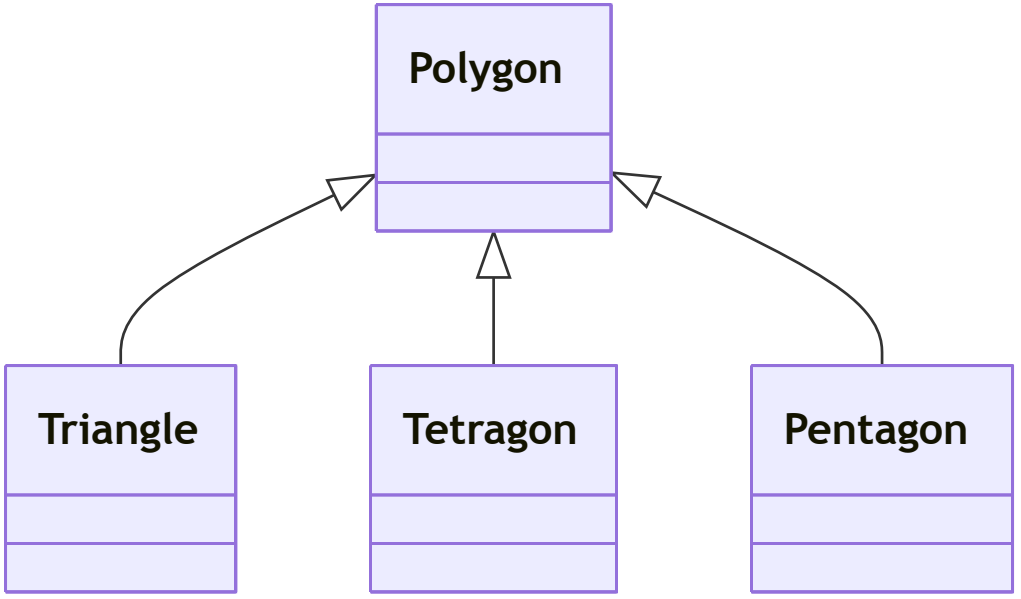
```
class Line:
    def __init__(self, start, end):
        self.start = start
        self.end = end

    def length(self):
        return
self.start.distance(self.end)
```

Inheritance

- Similar objects often share similar properties and methods. For example, all geometric elements consist of points that are connected by lines.
- To avoid having to redefine these attributes and methods each time, and potentially make mistakes in the process, there is *inheritance* in OOP.
- OOP allows defining specialized *subclasses*, or generalized *superclasses*.
- The subclass (specialized class) possesses the methods and attributes of the generalized superclass (base class).
- Also called *inheritance*, inspired by genetics.

Example Inheritance



Polymorphism

- An object of a *derived class* can always also be regarded as an instance of the *base class*.
- The derived class can *override* the inherited methods/attributes of the *base class* and thus redefine them.
- In statically typed languages, moreover: A variable that can hold an object of the *base class* can also store an object of a *derived class*.

Data Encapsulation

- The goal of *Datenkapselung* is to have control over which attributes are readable or writable and how this can be achieved
- Furthermore one controls whether attributes are only *readable* or also *writable*. For this, attributes are declared as *private* and are then readable only through getter functions and modifiable through setter functions.
- For this, one declares attributes or methods as:
  - *private* – Only the instance of the same class can access
  - *protected* – Only the instance of the same class or a subclass can access
  - *public* – Everyone can access

Dynamic behavior in classes

- In OOP it is assumed that classes are *static* and there are no attributes or methods other than those defined in the class.
- Python offers a higher degree of dynamism than other programming languages:
  - Dynamic typing of variables
  - Dynamic binding of methods and attributes
  - We can even add attributes and methods ourselves if they are not defined in the class.
  - Thus, for example, at runtime we can create references between objects that have no relation to the predefined data model.

Object-Oriented Programming Paradigms

Inheritance	Generalization	Polymorphism	Encapsulation
Attributes and methods from parent classes are inherited by child classes. This helps avoid redundancies and errors.	Commonalities are implemented in generalized parent classes.	Child classes can override methods and thus redefine them.	Encapsulation of data and methods in objects is a protective mechanism to prevent faulty changes.

Unified Modeling Language - Fundamentals

- *UML (Unified Modeling Language)* is an ISO standard for describing software designs
- It is used for the design, procurement, implementation, and documentation of:
  - functionalities
  - structures
  - business processes
  - user interactions

The standard comprises 13 diagram types, the most common:

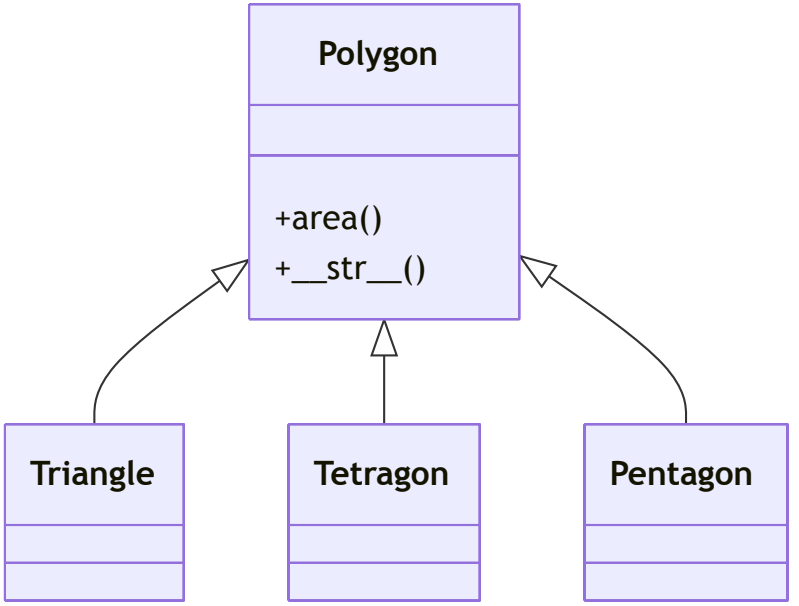
- *Class diagrams* (For class and data structures)
- *Activity diagrams* (For general processes and algorithms)
- *Sequence diagrams* (For interactions)
- *Use-case diagrams* (For users and use cases)

UML Class Diagram

- The *most commonly used diagram* in object-oriented modeling
- In a class diagram, the following are modeled:
  - Classes with: attributes, methods
  - Relationships
  - Hierarchies and inheritances

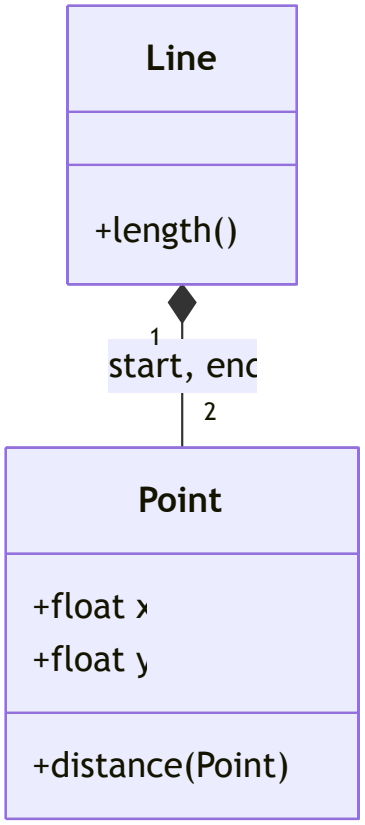
Class Diagram – Inheritance

- *Inheritance* is drawn in UML by a reference with a *filled triangle* "▲" at the superclass.
- To illustrate, for example, that `Triangle`, `Tetragon`, and `Pentagon` are subclasses of the Polygon, we can draw:



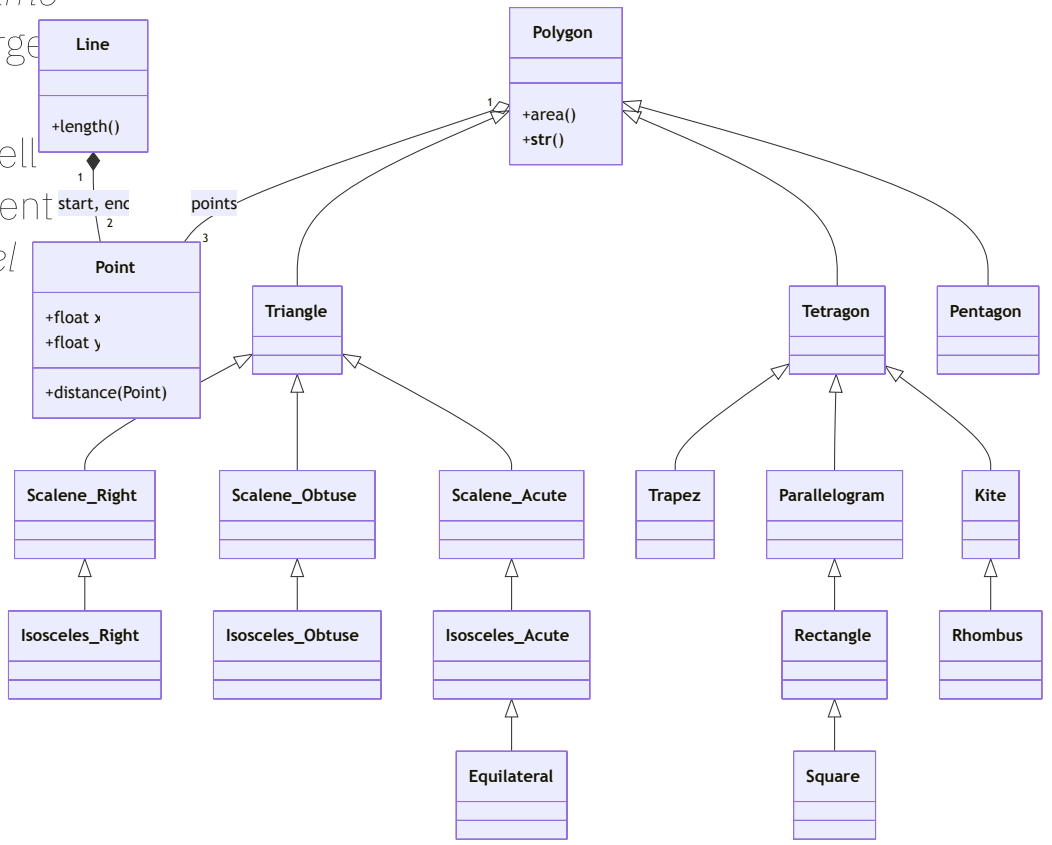
Class Diagram - References

- *References* are indicated in UML with lines between objects. The type of line and the arrow indicate the type of reference.
- In UML, references are distinguished by the type of *ownership*:
  - *Aggregated* - an object is part of another and can exist without it
  - *Composition* - cannot exist without this
  - *Association* - completely independent
- *Numbers* on the references indicate the *multiplicity*, i.e., how many objects are involved in this relation.



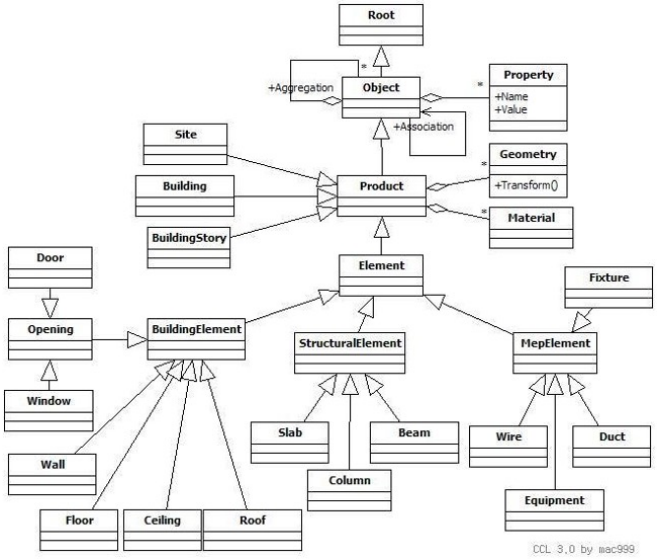
Class Diagram - Complete Example

- *Klassendiagramme* can get quite large fairly quickly
- They are very well suited to represent the *(data) model*



Application Example – BIM (Building Information Models)

- Construction drawings are nowadays stored as *IFC (Industry Foundation Classes)*
- *IFC* is an object-oriented model, with inheritance, specialization, generalization, and polymorphism
- It is, among other things, documented in *UML*



Design Approach

Midjourney: Software Waterfall

Software Design

- Software is rarely developed alone, but often in a team over a longer period.
- For this, a project plan is needed that describes how the software should be developed in a division-of-labor fashion.
- Creating this project plan is called *Software Design*.
- The process closely resembles the *design methodology in environmental and civil engineering*.
- The project plan comprises:
  - the *class design* (blueprint)
  - the *programming approach* (construction scheduling)

Software Design - Phases

- *Requirements definition* – Definition of which functions and constraints are to be implemented
- *Design*
  - Software class design
  - Implementation planning – What will be done when, by whom, and how it will be implemented
- *Execution*
  - Implementation – Programming of the individual classes and modules
  - Integration – Bringing the modules together into the finished solution
- *Acceptance* – Testing of the finished software product
  - Module testing – Unit testing of modules
  - Integration testing – Testing the integration of modules
  - System testing – Testing the complete set of modules

Comparison: Software Engineering vs. Civil Engineering

Software design

- *Requirements definition:* Definition of which functions and constraints are implemented

- *Design*
  - Module design – Definition of which parts the software has
  - Class design – Definition of how the software is structured into classes
  - Implementation planning – What will be implemented when, by whom, and how

Civil Engineering

- *Tender specification:* Documenting which functions and constraints the building must satisfy
- *Design*
  - Architecture and high-level design – Definition of how the building is roughly structured
  - Detail design and trades planning - Definition of how parts and the trades are implemented
  - Execution planning – What will be built when, by whom, and how built

Comparison: Software Engineering vs. Civil Engineering (Continued)

Software Design (Continued)

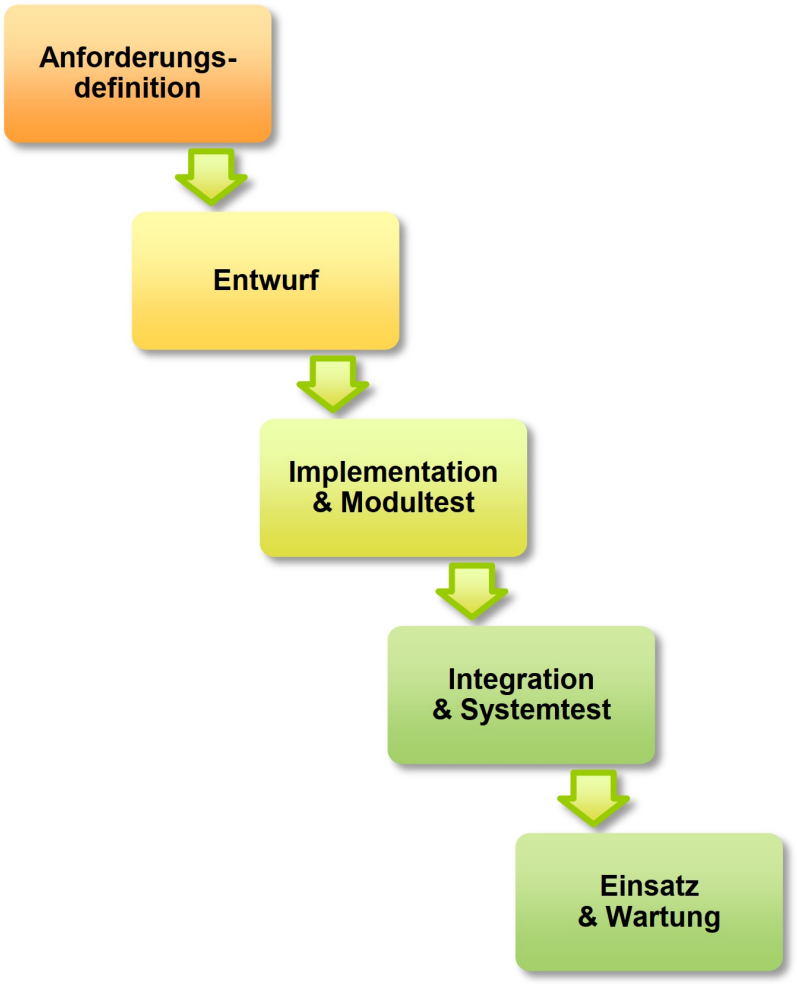
- *Execution*
  - Implementation – programming the individual classes & modules
  - Integration – bringing the modules together into the finished solution
- *Acceptance*
  - Module test – unit testing of modules
  - Integration test – testing the combination of modules
  - System test – testing the entirety of the modules

Civil Engineering (Continued)

- *Execution*
  - Individual trades are implemented
  - Integrate trades in construction, such as integrating energy and water with climate control
- *Acceptance*
  - Acceptance of individual trades
  - Test several trades together
  - Handover of the entire building

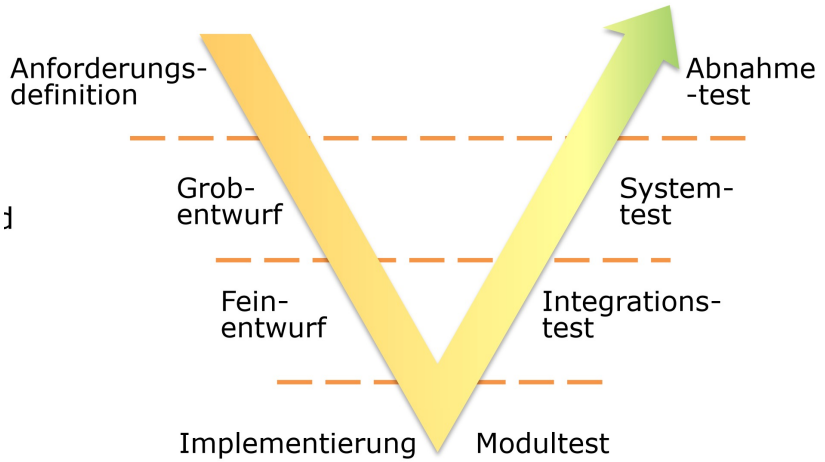
Linear Method - Waterfall Method

- Traditional model that is still frequently required in tenders for large systems
- Development is divided into several sequential steps, and each step must be completed before the next
- User involvement only in the requirements definition
- Each activity is documented → Well suited for tenders (after the requirements definition or the design, ISO 9000)



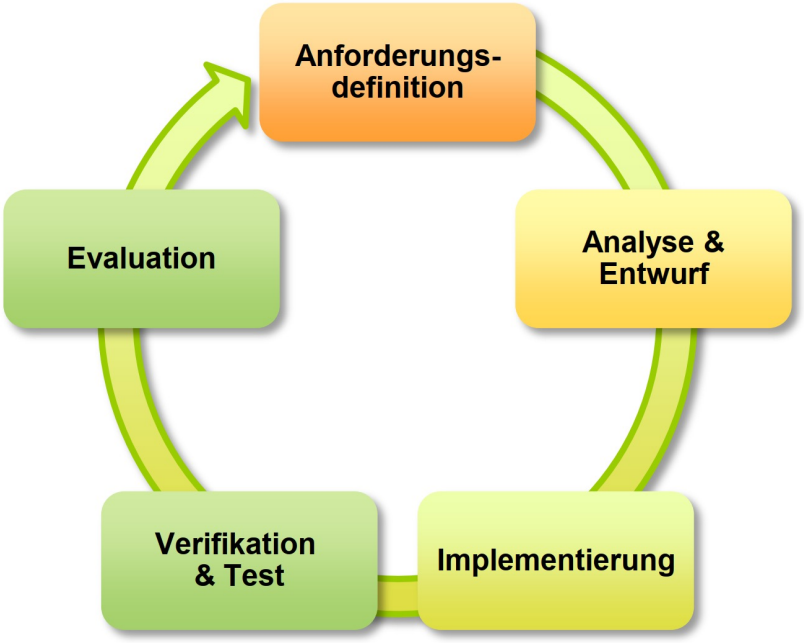
V-Model

- Primarily used in the development of safety-critical software (cars, aircraft, etc.)
- Separates development (left side) and testing activities (right side)
- Tests are defined early in the development phase
- Goal: high test coverage
- User involvement in requirements definition and in acceptance testing



Agile Method

- Modern method to adapt to constantly changing requirements
- The goal is incremental development of the solution to keep the effort and the complexity of individual steps in check
- Starts with a simple and extensible implementation (MVP – Minimum Viable Product)
- Incremental expansion and improvement of the product with regular releases (usually every 3 months)
- Design flaws in the early iterations can lead to a complete redesign



Questions?

PROGRAMMIERUNG UND DATENBANKEN